

NumPy از صفر با Machine Learning پروژه: پیاده‌سازی الگوریتم‌های

هدف کلی

هدف این پروژه درک عمیق:

- Machine Learning ریاضیات الگوریتم‌های
- نحوه آموزش مدل‌ها
- Gradient Descent
- محاسبه فاصله‌ها
- Vectorization در NumPy
- Prediction
- معیارهای ارزیابی
- Visualization
- ساختاردهی و مستندسازی کد

قوانین پروژه

- ❌ استفاده از `sklearn.linear_model`
- ❌ استفاده از `sklearn.neighbors`
- ❌ استفاده از `sklearn.metrics`
- ❌ Machine Learning استفاده از هر مدل آماده
- ❌ استفاده از `scipy` برای محاسبات مدل
- ✅ برای تمام محاسبات NumPy
- ✅ فقط برای رسم Seaborn یا Matplotlib
- ⚠️ فقط برای ساخت دیتاست مصنوعی مجاز است `sklearn.datasets` استفاده از
- پیاده‌سازی شوند **From Scratch** تمام الگوریتم‌ها باید
- داشته باشد Docstring و Type Hinting و PEP8 کد باید

Linear Regression From Scratch — فاز اول

مرحله 1 — آماده‌سازی محیط

کن import ابتدا کتابخانه‌های موردنیاز را:

```
import numpy as np
import matplotlib.pyplot as plt
```

برای ساخت دیتاست نمونه، در صورت نیاز:

```
from sklearn.datasets import make_regression
```

فقط برای تولید داده استفاده می‌شود، نه برای ساخت مدل sklearn.

مرحله 2 — ساخت دیتاست

مصنوعی ایجاد کن Regression یک دیتاست

مثلاً:

```
X, y = make_regression(  
    n_samples=200,  
    n_features=1,  
    noise=20,  
    random_state=42  
)
```

کارهایی که باید انجام دهی

1. تعداد نمونه‌ها را مشخص کن.
 2. تعداد ویژگی‌ها را مشخص کن.
 3. اضافه کن noise مقداری.
 4. تعیین کن تا نتایج قابل تکرار باشند random_state.
- سپس شکل داده‌ها را بررسی کن:

```
print(X.shape)  
print(y.shape)
```

Train و Test مرحله 3 — تقسیم داده به

انجام بدهیم NumPy استفاده کنیم، تقسیم داده را خودمان با train_test_split چون اجازه نداریم از

مثلاً:

```
np.random.seed(42)  
  
indices = np.random.permutation(len(X))
```

```
split_index = int(0.8 * len(X))

train_indices = indices[:split_index]
test_indices = indices[split_index:]

X_train = X[train_indices]
X_test = X[test_indices]

y_train = y[train_indices]
y_test = y[test_indices]
```

هدف این مرحله

داده را به

- آموزش 80%
- تست 20%

تقسیم کن.

LinearRegressionFromScratch مرحله 4 — طراحی کلاس

کلاسی با نام زیر بساز

```
class LinearRegressionFromScratch:
    ...
```

کلاس باید حداقل این قسمت‌ها را داشته باشد

```
__init__()
fit()
predict()
mse()
r2_score()
```

Hyperparameterها مرحله 5 — تعریف

دریافت کن Hyperparameter دو `__init__` در

```
learning_rate  
epochs
```

مثلاً:

```
model = LinearRegressionFromScratch(  
    learning_rate=0.01,  
    epochs=1000  
)
```

وظیفه

مقدارهای زیر را داخل شیء ذخیره کن:

```
self.learning_rate  
self.epochs
```

داشته باشی Bias همچنین باید برای مدل جایی برای ذخیره وزن ها و

مثلاً:

```
self.weights  
self.bias  
self.loss_history
```

Bias و Weight مرحله 6 — مقداردهی اولیه

قبل از آموزش باید پارامترهای مدل را مقداردهی کنی.

مدل Linear Regression:

$$\hat{y} = Xw + b$$

پس باید داشته باشیم:

```
weights  
bias
```

برای مثال:

```
self.weights = np.zeros(X.shape[1])  
self.bias = 0.0
```

fit داخل Prediction مرحله 7 — پیاده سازی

باید مقدار پیش بینی شده را محاسبه کنی Epoch در هر

$$\hat{y} = Xw + b$$

در NumPy:

```
y_pred = X @ self.weights + self.bias
```

نکته مهم

از حلقه روی تک تک نمونه ها استفاده نکن

✗:

```
for i in range(len(X)):
    ...
```

استفاده کن Vectorization برای محاسبات اصلی از

Loss مرحله 8 — محاسبه

استفاده کن Mean Squared Error از Linear Regression برای

$$MSE = \frac{1}{n} \sum (y - \hat{y})^2$$

در NumPy:

```
error = y_pred - y
loss = np.mean(error ** 2)
```

را در Epoch هر Loss مقدار

```
self.loss_history
```

ذخیره کن.

Gradient Descent مرحله 9 — پیاده‌سازی

Gradient مربوط به Weight:

$$\frac{\partial L}{\partial w} = \frac{2}{n} X^T (\hat{y} - y)$$

Gradient مربوط به Bias:

$$\frac{\partial L}{\partial b} = \frac{2}{n} \sum (\hat{y} - y)$$

در NumPy:

```
dw = (2 / n_samples) * (X.T @ error)
db = (2 / n_samples) * np.sum(error)
```

کن Update سپس پارامترها را:

```
self.weights -= self.learning_rate * dw
self.bias -= self.learning_rate * db
```

این کار را به تعداد

```
epochs
```

تکرار کن.

fit مرحله 10 — تکمیل متد

باید در پایان fit

1. مقداردهی شوند Weightها.
2. مقداردهی شود Bias.
3. محاسبه شود Prediction.

4. محاسبه شود Error.
5. محاسبه شود Loss.
6. محاسبه شود Gradient.
7. آپدیت شوند Bias و Weight.
8. ذخیره شود Loss.

در نهایت:

```
return self
```

predict مرحله 11 — پیاده سازی

متد:

```
predict(X)
```

آموزش دیده استفاده کند Bias و Weight باید فقط از

فرمول:

$$\hat{y} = Xw + b$$

بنابراین:

```
return X @ self.weights + self.bias
```

MSE مرحله 12 — پیاده سازی

بساز MSE یک متد برای

```
mse(y_true, y_pred)
```

فرمول:

$$MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

نباید از:

```
sklearn.metrics.mean_squared_error
```

استفاده کنی.

R² Score مرحله 13 — پیاده‌سازی

فرمول:

$$R^2 = 1 - \frac{\sum (y - \hat{y})^2}{\sum (y - \bar{y})^2}$$

بنویسی NumPy در این مرحله باید این معیار را خودت با

از:

```
sklearn.metrics.r2_score
```

استفاده نکن.

مرحله 14 — آموزش مدل

حالا مدل را بساز:

```
model = LinearRegressionFromScratch(  
    learning_rate=0.01,  
    epochs=1000  
)
```

سپس:

```
model.fit(X_train, y_train)
```


مرحله 15 — Prediction روی Test

```
y_pred = model.predict(X_test)
```

بعد بررسی کن که

```
y_test  
y_pred
```

شکل مناسبی دارند.

مرحله 16 — ارزیابی مدل

را محاسبه کن MSE:

```
MSE
```

و سپس:

```
R2
```

در خروجی چیزی شبیه این داشته باش:

```
MSE: ...  
R2 Score: ...
```

Regression Line مرحله 17 — رسم

از داده‌ها رسم کن Scatter یک نمودار

باید:

- نقاط واقعی داده‌ها نمایش داده شوند.
- روی آن‌ها نمایش داده شود Regression خط

برای این کار از:

```
plt.scatter(...)\nplt.plot(...)
```

استفاده کن.

هدف

نشان بده که مدل خطی یادگرفته شده چگونه داده‌ها را تقریب می‌زند.

Loss Curve مرحله 18 — رسم

از:

```
model.loss_history
```

استفاده کن.

X: محور

Epoch

Y: محور

MSE Loss

هدف نمودار:

Gradient Descent در طول فرآیند Loss نشان دادن کاهش

Linear Regression مرحله 19 — بررسی

در پایان فاز اول بررسی کن

- کاهش پیدا کرده؟ Loss آیا
- منطقی است؟ Regression Line آیا
- مناسب است؟ MSE آیا
- به مقدار مناسبی رسیده؟ R^2 آیا
- باعث بهبود آموزش شده؟ Epoch آیا افزایش
- خیلی بزرگ یا خیلی کوچک نیست؟ Learning Rate آیا

KNN From Scratch — فاز دوم

Classification مرحله 20 — ساخت دیتاست

یک دیتاست دو بعدی بساز

مثلاً:

```
from sklearn.datasets import make_classification

X, y = make_classification(
    n_samples=300,
    n_classes=3,
    n_features=2,
    class_std=1.8,
    random_state=42
)
```

فقط برای تولید داده استفاده می‌شود sklearn دقت کن که اینجا

KNN مرحله 21 — تقسیم داده

تقسیم کن Test و Train به NumPy داده را با

مثلاً:

```
80% Train
20% Test
```

در این پروژه بهتر است تقسیم داده را خودت انجام دهی و از

```
train_test_split
```

استفاده نکنی.

KNNFromScratch مرحله 22 — طراحی کلاس

کلاس:

```
class KNNFromScratch:  
    ...
```

حداقل باید داشته باشد:

```
__init__()  
fit()  
predict()  
accuracy_score()
```

K مرحله 23 — تعریف

را دریافت کن k مقدار `__init__` در

```
model = KNNFromScratch(k=7)
```

و ذخیره کن:

```
self.k = k
```

fit مرحله 24 — پیاده سازی

نیست Weight آموزش به معنی محاسبه، Linear Regression برخلاف KNN در

را ذخیره کن Train فقط داده های fit در

```
X_train  
y_train
```

مثلاً:

```
self.X_train = X  
self.y_train = y
```

محاسبه — مرحله 25 Euclidean Distance

فرمول فاصله اقلیدسی:

$$d(x_1, x_2) = \sqrt{\sum_j (x_{1j} - x_{2j})^2}$$

محاسبه کنی **Vectorized** اما مهم‌ترین قسمت پروژه این است که فاصله را

استفاده کنی broadcasting می‌توانی به شکلی مشابه از Train و Test برای مجموعه‌ای از داده‌های

```
X_test[:, np.newaxis] - X_train
```

در نتیجه ماتریسی از اختلاف‌ها ایجاد می‌شود.

سپس:

```
square  
→ sum  
→ sqrt
```

همسایه نزدیک K مرحله 26 — پیدا کردن

بعد از محاسبه فاصله‌ها باید نزدیک‌ترین نمونه‌ها را پیدا کنی

از:

```
np.argsort(...)
```

استفاده کن.

هدف:

```
Test برای هر نمونه  
↓  
Trainها فاصله تمام  
↓  
مرتب‌سازی فاصله‌ها  
↓  
مورد اول K انتخاب
```

همسایه K مرحله 27 — رأی‌گیری بین

مثلاً اگر:

K = 7

و کلاس همسایه‌ها:

[1, 1, 2, 1, 3, 2, 1]

باشد:

Class 1 → 4 رأی

Class 2 → 2 رأی

Class 3 → 1 رأی

Prediction: پس

1

است.

predict مرحله 28 — تکمیل

متد:

`predict(X)`

باید:

1. فاصله‌ها را محاسبه کند.
2. همسایه نزدیک را پیدا کند K.
3. همسایه‌ها را بردارد Label.
4. Majority Vote دهد.
5. Prediction برگرداند.

خروجی باید چیزی شبیه

[0, 1, 2, 1, 0, ...]

باشد.

Accuracy مرحله 29 — پیاده سازی

یک متد:

```
accuracy_score(y_true, y_pred)
```

بنویس.

فرمول:

$$Accuracy = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

می توانی از مقایسه ی آرایه ها استفاده کنی NumPy با

```
np.mean(y_true == y_pred)
```

مثلاً اگر:

```
y_true = [0, 1, 2, 1]
y_pred = [0, 1, 1, 1]
```

سه مورد درست است:

```
3 / 4 = 0.75
```

پس:

```
Accuracy = 0.75
```

KNN مرحله 30 — آموزش و تست

مدل را بساز:

```
k = 7

knn_model = KNNFromScratch(k=k)
```

سپس:

```
knn_model.fit(X_train, y_train)
```

Prediction: و

```
y_pred_test = knn_model.predict(X_test)
```

Accuracy مرحله 31 — محاسبه

```
acc = knn_model.accuracy_score(  
    y_test,  
    y_pred_test  
)
```

و چاپ:

```
K = 7  
Accuracy = ...
```

Decision Boundary مرحله 32 — رسم

است KNN این یکی از مهم‌ترین بخش‌های پروژه.

چون دیتاست

2 features

دارد، می‌توانیم صفحه دوبعدی بسازیم.

مراحل:

1. را پیدا کن X محدوده
2. را پیدا کن Y محدوده
3. ایجاد کن Grid یک
4. بده KNN را به Grid هر نقطه
5. کلاس پیش‌بینی‌شده را برای هر نقطه بگیر

داده‌های واقعی را در آن قرار ده 7

```
np.meshgrid(...)
```

```
plt.contourf(...)
```

K = 1
K = 3
K = 5
K = 7
K = 15

K = 1 → Accuracy = ...
K = 3 → Accuracy = ...
K = 5 → Accuracy = ...
K = 7 → Accuracy = ...
K = 15 → Accuracy = ...

ها K مرحله 34 — مقایسه

در این قسمت توضیح بده

خیلی کوچک باشد K اگر

مدل حساس‌تر به داده‌های اطراف است و ممکن است

Overfitting

رخ دهد.

خیلی بزرگ باشد K اگر

مدل بیش از حد تحت تأثیر تعداد زیادی از نمونه‌ها قرار می‌گیرد و ممکن است

Underfitting

رخ دهد.

روی دیتاست تو عملکرد بهتری داشته است K در نهایت توضیح بده که کدام

مرحله 35 — تمیزکاری کد

قبل از تحویل، کد را بررسی کن.

هر کلاس باید:

- نام مناسب داشته باشد.
- داشته باشد Type Hint.
- داشته باشد Docstring.
- نام متغیرها واضح باشد.
- خطوط خیلی طولانی نداشته باشد.
- کد تکراری نداشته باشد.
- باشند Vectorized محاسبات اصلی.

ساختار کلی پیشنهادی:

```
ML From Scratch
|
├─ Imports
|
├─ Linear Regression
|   ├── Dataset
|   ├── Train/Test Split
|   └─ LinearRegressionFromScratch
```

```

|   |— Training
|   |— Prediction
|   |— MSE
|   |— R2
|   |— Regression Plot
|   |— Loss Curve
|
|— KNN
   |— Dataset
   |— Train/Test Split
   |— KNNFromScratch
   |— Euclidean Distance
   |— Prediction
   |— Accuracy
   |— Decision Boundary
   |— K Comparison

```

مرحله 36 — تست نهایی

قبل از تحویل، این موارد را بررسی کن:

Linear Regression

- ☐ ساخته شده LinearRegressionFromScratch کلاس
- ☐ learning_rate قابل تنظیم است
- ☐ epochs قابل تنظیم است
- ☐ fit پیادهسازی شده
- ☐ Gradient Descent پیادهسازی شده
- ☐ predict پیادهسازی شده
- ☐ از صفر نوشته شده MSE
- ☐ از صفر نوشته شده R²
- ☐ Loss History ذخیره شده
- ☐ Regression Line رسم شده
- ☐ Loss Curve رسم شده

KNN

- ☐ ساخته شده KNNFromScratch کلاس
- ☐ K قابل تنظیم است
- ☐ fit پیادهسازی شده
- ☐ Euclidean Distance پیادهسازی شده
- ☐ Distance به صورت Vectorized محاسبه شده
- ☐ نزدیکترین همسایه پیدا می‌شود K
- ☐ Majority Vote پیادهسازی شده

- ☐ predict پیاده‌سازی شده
- ☐ Accuracy از صفر نوشته شده
- ☐ Decision Boundary رسم شده
- ☐ آزمایش شده K چند مقدار

قوانین کلی

- ☐ استفاده نشده sklearn مدل آماده
- ☐ استفاده نشده sklearn آماده Metric
- ☐ انجام شده NumPy محاسبات با
- ☐ دارد Type Hint کد
- ☐ دارد Docstring
- ☐ است PEP8 کد
- ☐ مشخص هستند Notebook خروجی‌ها در